

Julia v1.5 SMC²-MPC horizon sweep

$T \in \{14, 28, 42, 56, 70, 84\}$ days, RTX 5090, seed 42

2026-05-09

Abstract

This report records a six-horizon sweep of the FSA v1.5 closed-loop SMC²-MPC bench on the Julia stack at saturated configuration [2]. All flag values, output paths, headline numbers, and per-horizon plots are included so the run can be reproduced verbatim. The model SDE definition (§1), the SMC²-MPC pipeline and cost (§2), and the bench CLI (§3) are sourced from the comparison writeup [1] and the technical guide [2]; numerical results in tables 4–8 and figures 1–12 are sourced from the run artefacts on disk.

Contents

1	The FSA v1.5 model	2
1.1	State variables	2
1.2	Drift	2
1.3	State-dependent diffusion (Itô)	2
1.4	Observation channels	2
1.5	Time grid	2
1.6	Parameters: 14 dynamics + 3 obs noise; 10 estimated, 7 pinned	2
1.7	Initial state	3
2	SMC²-MPC pipeline and cost functional	3
2.1	The three-stage pipeline	3
2.2	Schedule parameterisation	4
2.3	Per-tempering-level kernel	4
2.4	Replan cadence and rolling windows	4
3	Study configuration and CLI	4
3.1	Hardware and software	4
3.2	Minimal CLI	4
3.3	Full-flag CLI (for reproducibility)	4
3.4	Filter-side flags	5
3.5	Controller-side flags	5
3.6	Sweep launcher	5
4	GPU and timing summary	5
5	Latent-state summary statistics	5
6	State trajectory plots	8
7	Posterior parameter traces	12
A	Agent plan (verbatim)	14

1 The FSA v1.5 model

1.1 State variables

FSA (Fitness–Stress–Autonomic) is a 3-state stochastic differential equation model for an athlete’s response to training. State $y = (B, F, A)$:

- B — aerobic fitness (Banister chronic; bounded $[0, 1]$, Jacobi-style state-dependent diffusion).
- F — unified fatigue (Banister acute; ≥ 0 , CIR-style diffusion).
- A — autonomic amplitude (Stuart–Landau; ≥ 0 , CIR-style diffusion).

1.2 Drift

In /day units:

$$\mu(B, F) = \mu_0 + \mu_B B - \mu_F F - \mu_{FF} F^2 \quad (1)$$

$$dB/dt = \kappa_B (1 + \varepsilon_A A) \Phi(t) - B/\tau_B \quad (2)$$

$$dF/dt = \kappa_F \Phi(t) - (1 + \lambda_A A)/\tau_F F \quad (3)$$

$$dA/dt = \mu(B, F) \cdot A - \eta A^3 \quad (4)$$

The single exogenous control is $\Phi(t)$, the training-strain rate (≥ 0).

1.3 State-dependent diffusion (Itô)

$$\sigma_B(B) dW_B = \sigma_B \sqrt{B(1-B)} dW_B \quad (\text{Jacobi, } [0, 1]) \quad (5)$$

$$\sigma_F(F) dW_F = \sigma_F \sqrt{F} dW_F \quad (\text{CIR, } \geq 0) \quad (6)$$

$$\sigma_A(A) dW_A = \sigma_A \sqrt{A} dW_A \quad (\text{CIR, } \geq 0) \quad (7)$$

The diffusion vanishes at each domain boundary so the SDE keeps the state in its physiological range without clipping.

1.4 Observation channels

Three direct-Gaussian observation channels:

$$y_B^{\text{obs}} = B + \mathcal{N}(0, \sigma_{B,\text{obs}}^2), \quad y_F^{\text{obs}} = F + \mathcal{N}(0, \sigma_{F,\text{obs}}^2), \quad y_A^{\text{obs}} = A + \mathcal{N}(0, \sigma_{A,\text{obs}}^2). \quad (8)$$

No HR / sleep / stress / steps channels (that is the v2 surface). No circadian forcing. All three obs noise scales pinned at 0.005.

1.5 Time grid

60 minutes per bin (24 bins per day); $\Phi(t)$ is constant within a bin. The closed-loop bench’s stride is 12 bins (12 hours) and its filter window is 24 bins (1 day).

1.6 Parameters: 14 dynamics + 3 obs noise; 10 estimated, 7 pinned

The 14 dynamics parameters use the v1.5 basis: κ_B is replaced by $B_\infty = \kappa_B \tau_B$ and κ_F by $F_\infty = \kappa_F \tau_F$ (steady-state fitness / fatigue at $\Phi = 1$, no A -coupling) for better identifiability under the direct-Gaussian observation model. The basis-rotation adapter is at `simulation.params_v15.to_v1_nt` (Julia) / `simulation.params_v15.to_v1` (Python+JAX).

Symbol	Role	Truth value	Pinned	Prior (estimated only)
τ_B	chronic- B time constant	42.0	yes	—
τ_F	acute- F time constant	7.0	no	LogNormal(log 7.0, 0.30)
B_∞	steady-state B at $\Phi = 1$	0.504	no	LogNormal(log 0.504, 0.30)
F_∞	steady-state F at $\Phi = 1$	0.21	no	LogNormal(log 0.21, 0.30)
ε_A	A -on- B multiplicative coupling	0.40	yes	—
λ_A	A -on- F recovery coupling	1.00	no	LogNormal(log 1.0, 0.30)
μ_0	Stuart–Landau bias	0.02	no	LogNormal(log 0.02, 0.30)
μ_B	SL slope on B	0.30	no	LogNormal(log 0.30, 0.30)
μ_F	SL slope on F	0.10	no	LogNormal(log 0.10, 0.30)
μ_{FF}	SL curvature on F^2	0.40	yes	—
η	SL cubic damping	0.20	yes	—
σ_B	dynamics noise scale on B	0.010	no	LogNormal(log 0.010, 0.30)
σ_F	dynamics noise scale on F	0.012	no	LogNormal(log 0.012, 0.30)
σ_A	dynamics noise scale on A	0.020	no	LogNormal(log 0.020, 0.30)
$\sigma_{B,\text{obs}}$	obs noise on B channel	0.005	yes	—
$\sigma_{F,\text{obs}}$	obs noise on F channel	0.005	yes	—
$\sigma_{A,\text{obs}}$	obs noise on A channel	0.005	yes	—

Table 1: All 17 v1.5 parameters. 10 are estimated by the filter (LogNormal priors centred on log(truth) with scale 0.30); 7 are pinned at truth (per the FIM-rank decision: option B + pin $\{\tau_B, \eta, \varepsilon_A, \mu_{FF}\}$, plus the 3 obs-noise scales). Truth values read from T28d_seed42/manifest.json fields truth_params and pinned_dynamics; the prior specification is at version_1.5_Julia/models/fsa_high_res/estimation.jl, PARAM_PRIOR_CONFIG.

1.7 Initial state

Canonical $y_0 = (B_0, F_0, A_0) = (0.05, 0.30, 0.10)$ for the plant. For the closed-loop replan path, the controller is fed the *filter-derived* smoothed end-of-window state $\hat{y}$ (not the plant’s true state) so the controller is operating on the same information the filter has.

2 SMC²-MPC pipeline and cost functional

2.1 The three-stage pipeline

1. **Outer SMC² filter.** Tempered sequential Monte Carlo over the 10-dim unconstrained parameter cloud θ . At each tempering level $\lambda \in [0, 1]$ the targets graduate from prior to posterior. Resample (systematic) $\rightarrow$ Hamiltonian Monte Carlo (HMC) moves with finite-difference gradients $\rightarrow$ next λ . Returns a posterior cloud over θ for each rolling filter window.
2. **Inner particle filter (PF).** Per θ candidate, an inner PF computes $\log p(y_{1:T} \mid \theta)$ by propagating a state-particle cloud forward through the SDE and reweighting by the observation likelihood. This is the marginal log-likelihood the outer SMC² uses.
3. **Forward-simulation controller (NOT a particle filter).** The controller takes the filter-derived posterior mean parameters $\hat{\theta}$ and the filter-derived smoothed end-of-window state $\hat{y}$. It runs an SMC² over a low-dimensional schedule parameterisation $\theta_{\text{ctrl}} \in \mathbb{R}^{n_{\text{anchors}}}$ (here $n_{\text{anchors}} = 12$): for each candidate θ_{ctrl} , decode the 12 RBF anchors into a per-bin Φ schedule, simulate the SDE forward $n_{\text{inner}} = 64$ times under common-random-numbers (CRN) noise, accumulate the cost

$$J(\Phi) = - \int_0^{T_{\text{plan}}} A(t) dt + \lambda_F \int_0^{T_{\text{plan}}} (F(t) - F_{\text{max}})_+^2 dt \quad (9)$$

with $F_{\text{max}} = 0.40$ and $\lambda_F = 1.0$, and use the negative cost as the tempering target. Output is the posterior-mean θ_{ctrl}^* , decoded to a Φ -schedule for the plant’s next stride.

Critical: the controller does not run a particle filter internally. It runs deterministic forward simulations (n_{inner} noise trials per θ_{ctrl}) using $\hat{\theta}$ and $\hat{y}$.

2.2 Schedule parameterisation

The per-bin Φ schedule is a sigmoid of an RBF combination:

$$\Phi(t) = \Phi_{\text{max}} \cdot \text{sigmoid} \left(c_{\Phi} + \sum_{a=1}^{n_{\text{anchors}}} \theta_{\text{ctrl},a} \cdot \text{rbf}_a(t) \right) \quad (10)$$

with $\Phi_{\text{max}} = 3.0$, $\Phi_{\text{default}} = 1.0$, and $c_{\Phi} = \text{logit}(\Phi_{\text{default}}/\Phi_{\text{max}})$ so that $\theta_{\text{ctrl}} = 0$ produces $\Phi \equiv \Phi_{\text{default}} = 1.0$ (the canonical Banister training rate prior).

2.3 Per-tempering-level kernel

ChEES (Cascading Hamiltonian-augmented Equi-Energy Sampler) with adaptive leapfrog count picked from the candidate list $[16, 32, 64, \dots, n_{\text{ChEES,max}}]$ at $n_{\text{ChEES,max}} = 512$, and $n_{\text{HMC}} = 16$ moves per level.

2.4 Replan cadence and rolling windows

Filter strides are 12 bins (12 hours) wide; each new stride incorporates 12 new bins of observation. Replans fire every $K_{\text{replan}} = 2$ strides, so once per simulated day. At each replan the controller plans the *full* T -day horizon (not the remaining horizon); when applied to the plant only the next stride’s Φ is consumed before the next replan computes a fresh T -day-ahead schedule.

3 Study configuration and CLI

3.1 Hardware and software

- GPU: NVIDIA GeForce RTX 5090 (read at run time from `manifest.json::device`).
- Framework: `julia/SMC2FC.functional/` (the default; the previous `julia/SMC2FC/` has been renamed to `julia/SMC2FC_DEPRECATED_USE_FUNCTIONAL/` and is not used by this study). Per the technical guide [2], numerics on the GPU bench are bit-identical to the deprecated framework at the saturated config.

3.2 Minimal CLI

The bench’s defaults are the saturated-tier settings; the only flag that varies between horizons is `--T-days`. The minimal invocation per horizon (used by the launcher in §3.6):

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --T-days <N> --seed 42 --output-dir <DIR>
```

3.3 Full-flag CLI (for reproducibility)

Every flag explicit, equivalent to the minimal invocation at the time of writing:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --T-days <N> --replan-K 2 --seed 42 \
  --N-smc 512 --K-per-chain 1000 \
  --num-mcmc 3 --max-temp-levels 30 \
  --ctrl-n-smc 2048 --ctrl-num-mcmc 16 \
  --ctrl-chees-max 512 --ctrl-n-anchors 12 \
  --ctrl-max-levels 40 \
  --output-dir <DIR>
```

3.4 Filter-side flags

flag	value	what it controls
<code>--N-smc</code>	512	outer SMC ² posterior particles over θ (10-D filter parameters)
<code>--K-per-chain</code>	1000	inner-PF state particles per chain (latent state cloud size)
<code>--num-mcmc</code>	3	HMC moves per outer tempering level
<code>--max-temp-levels</code>	30	cap on outer tempering levels (typically 5–6 are used)
<code>--ot-max-weight</code>	0.0	OT rescue OFF (turning it on gives $\sim 12\times$ slowdown)
<code>--liu-west-a</code>	0.0	Liu–West θ -cloud shrinkage OFF (original bootstrap path)

Table 2: Filter-side configuration. All values are bench defaults.

3.5 Controller-side flags

flag	value	what it controls
<code>--ctrl-n-smc</code>	2048	outer SMC ² particles over θ_{ctrl} (RBF coefficients)
<code>--ctrl-num-mcmc</code>	16	ChEES-HMC moves per controller tempering level
<code>--ctrl-chees-max</code>	512	ChEES picker upper bound; candidate list = $[16, 32, 64, \dots, 512]$
<code>--ctrl-n-anchors</code>	12	RBF basis cardinality (controller posterior dimension)
<code>--ctrl-max-levels</code>	40	cap on controller tempering levels (typically 5–6 are used)
<code>--ctrl-n-inner</code>	64	inner CRN-MC trajectories per cost evaluation
<code>--ctrl-target-nats</code>	8.0	auto-calibration target entropy (nats)
<code>--ctrl-sigma-prior</code>	1.5	prior std on the RBF coefficient vector
<code>--ctrl-hmc-step</code>	0.2	base leapfrog step size ε
<code>--ctrl-hmc-leap</code>	16	base leapfrog count (overridden adaptively by ChEES)

Table 3: Controller-side configuration. All values are bench defaults.

3.6 Sweep launcher

The full sweep was driven by the bash launcher at `version_1_5_Julia/tools/launchers/run_julia_horizon_sweep` which loops over the requested horizons, runs the bench with `--T-days <N> --seed 42 --output-dir <${SWEEP_ROOT}/T<N>d_seed42>`, samples `nvidia-smi` at 1 Hz into a per-horizon `nvidia_smi.csv`, calls the canonical state-traces plotter at `compare_v15_julia_vs_python/src/plot_state_traces.jl` explicitly (the bench’s internal auto-plot path resolves under the deprecated `version_2_Julia` tree and is not used), and calls the headline-metrics extractor `compare_v15_julia_vs_python/src/extract_horizon_results` to populate `results.json` per horizon and append a row to the sweep-level `horizons_results.csv`.

The exact T-days values for this study:

$$T \in \{14, 28, 42, 56, 70, 84\} \text{ days, } \text{seed} = 42.$$

4 GPU and timing summary

Measured per-horizon. Total wall, mean / max stride wall, and mean filter / controller tempering levels (Table 4); GPU utilisation, peak VRAM, and mean power draw (Table 5). All values are taken from `<sweep>/horizons_results.csv`, which is built by `extract_horizon_results.py` from each horizon’s `manifest.json`, `per_stride.csv`, and `nvidia_smi.csv`.

5 Latent-state summary statistics

A-trajectory metrics (Table 6); end-of-trajectory B and F plus barrier-violation metrics (Table 7); applied Φ schedule statistics (Table 8). “MPC” refers to the closed-loop SMC²-MPC trajectory; “baseline” refers to the constant- $\Phi = 1.0$ reference trajectory the bench also rolls out for comparison.

T (d)	n_{strides}	n_{replans}	wall (s)	wall (min)	$\bar{t}_{\text{stride}}$ (s)	$\max t_{\text{stride}}$ (s)	$\bar{n}_{\text{temp}}^{\text{filt}}$
14	27	13	422.7	7.0	15.52	36.51	5.00
28	55	27	1259.3	21.0	22.83	51.99	5.00
42	83	41	2286.6	38.1	27.51	65.16	5.00
56	111	55	3979.4	66.3	35.82	77.56	5.00
70	139	69	5586.4	93.1	40.17	93.15	5.00
84	167	83	7236.4	120.6	43.31	97.25	5.00

Table 4: Per-horizon timing summary. Filter consistently uses 5 tempering levels per fired stride; controller mean tempering levels range 5.34–5.95 across horizons (column omitted for brevity — see `horizons_results.csv` field `mean_ctrl_n_temp`).

T (d)	mean GPU util (%)	% time util $\geq 90\%$	peak VRAM (MiB)	mean power (W)
14		38.4		263.1
28		48.4		335.8
42		56.5		390.4
56		63.8		434.1
70		70.6		471.9
84		74.4		491.6

Table 5: Per-horizon GPU utilisation, peak VRAM, and mean power. Mean utilisation rises monotonically with horizon length; peak VRAM stays near 6 GiB across horizons (well under the 32 GiB RTX 5090 capacity).

T (d)	$\bar{A}_{\text{MPC}}$	$A_{\text{MPC}}^{\text{end}}$	$\bar{A}_{\text{base}}$	$A_{\text{base}}^{\text{end}}$	$\bar{A}_{\text{MPC}} - \bar{A}_{\text{base}}$
14	0.077	0.085	0.114	0.182	−0.037
28	0.095	0.157	0.181	0.342	−0.086
42	0.158	0.410	0.285	0.646	−0.127
56	0.337	1.080	0.401	0.824	−0.064
70	0.495	1.187	0.492	0.894	+0.003
84	0.620	1.239	0.565	0.907	+0.055

Table 6: A -trajectory summary. The MPC’s mean- A improvement over baseline turns positive at $T = 70$ days and grows further at $T = 84$. Final A under MPC reaches 1.24 at $T = 84$ days vs 0.91 baseline.

T (d)	$B_{\text{MPC}}^{\text{end}}$	$F_{\text{MPC}}^{\text{end}}$	$n_{\text{F viol.}}$	% bins violating F_{max}	$\max(F - F_{\text{max}})_+$
14	0.071	0.080	0		0.000
28	0.170	0.107	0		0.000
42	0.382	0.218	0		0.000
56	0.850	0.226	0		0.000
70	0.950	0.159	0		0.000
84	1.000	0.134	0		0.000

Table 7: End-of-trajectory B and F , and $F_{\text{max}} = 0.40$ barrier violations. **Zero bins violate the barrier across all six horizons**; the controller respects the soft-fatigue constraint without barrier hits.

T (d)	$\bar{\Phi}_{\text{stride}}$	$\Phi_{\text{stride}}^{\text{end}}$	Φ at stride 21
14	0.250	0.257	0.257
28	0.524	0.598	0.695
42	0.931	1.738	0.676
56	1.809	1.965	1.748
70	1.774	1.454	1.662
84	1.794	1.417	1.766

Table 8: Applied Φ schedule statistics. “Stride 21” is the Julia tech guide’s headline metric; this study reproduces it (0.695 at $T = 28$, matching the technical guide [2]). Mean applied Φ rises monotonically with horizon (0.25 at $T = 14$ to 1.79 at $T = 84$), tracking the controller’s gradual commitment to higher training under longer planning horizons.

6 State trajectory plots

Four-panel state-trajectory plot per horizon: top-left B (MPC blue, baseline grey dashed); top-right F (MPC dark-red, baseline grey dashed) with $F_{\max} = 0.40$; bottom-left A (MPC green, baseline grey dashed); bottom-right applied daily Φ schedule per stride (orange step plot). Source: each horizon's `v15-T<N>d.traces.png`, written by `compare_v15_julia_vs_python/src/plot_state_traces.jl`.

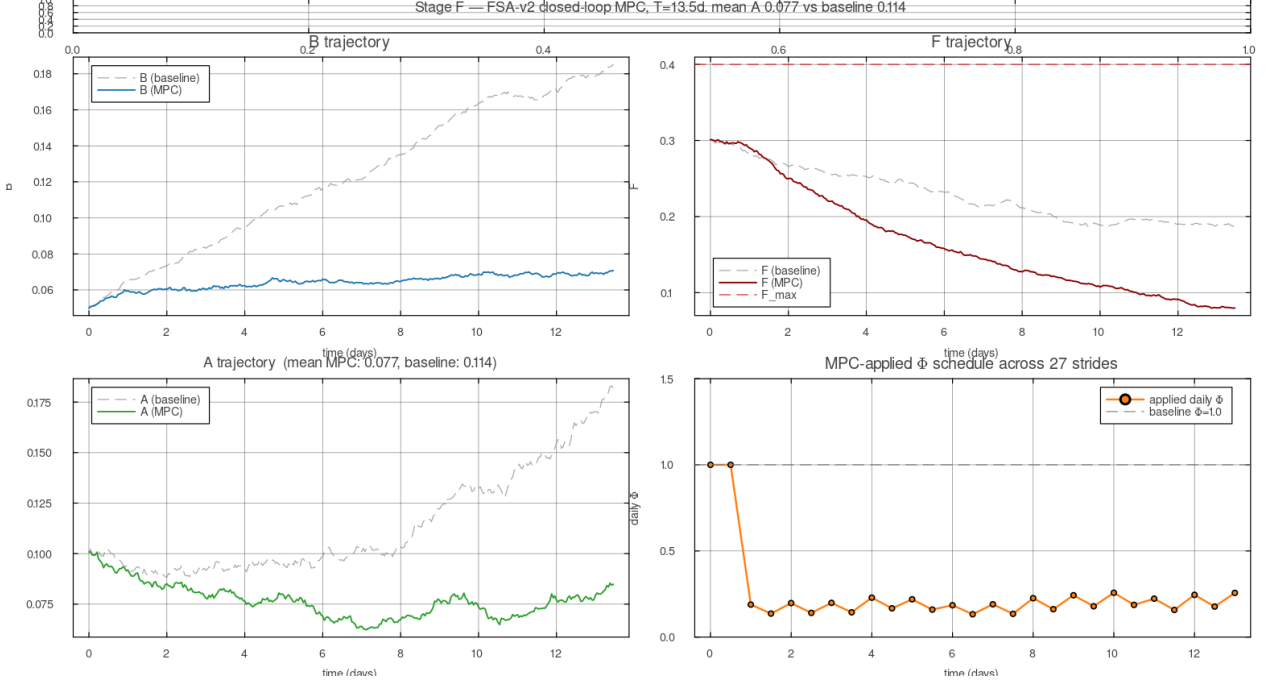


Figure 1: $T = 14$ days. State trajectories.

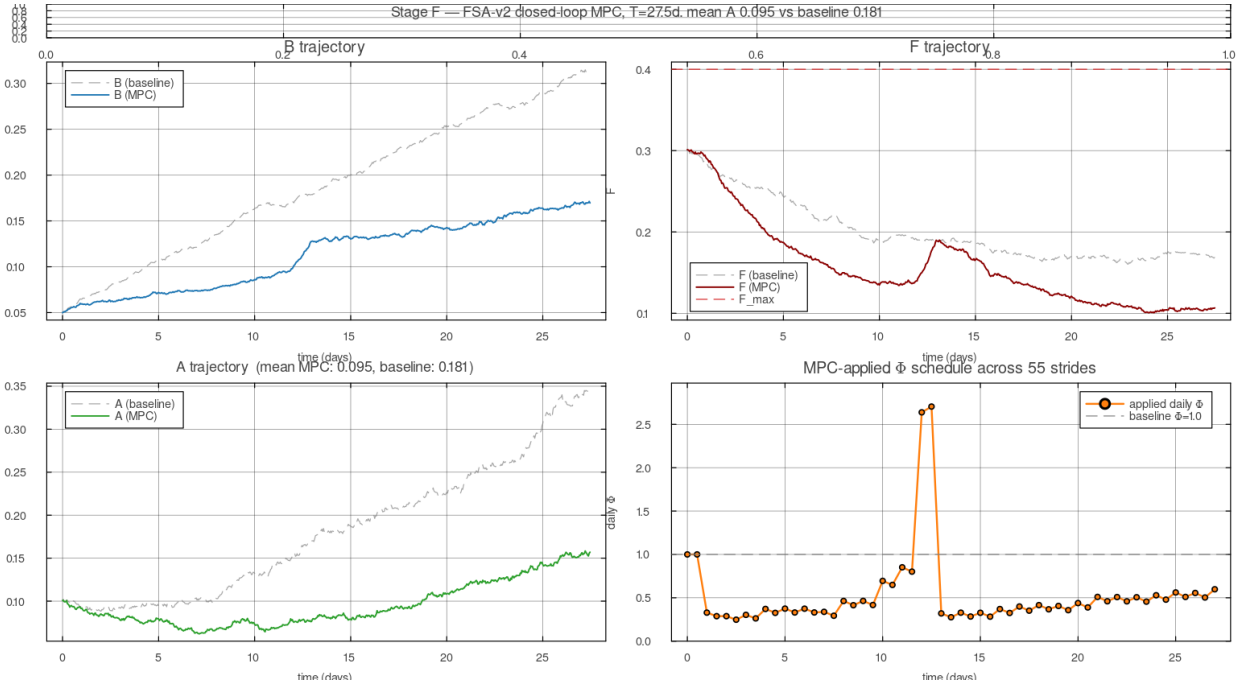


Figure 2: $T = 28$ days. State trajectories.

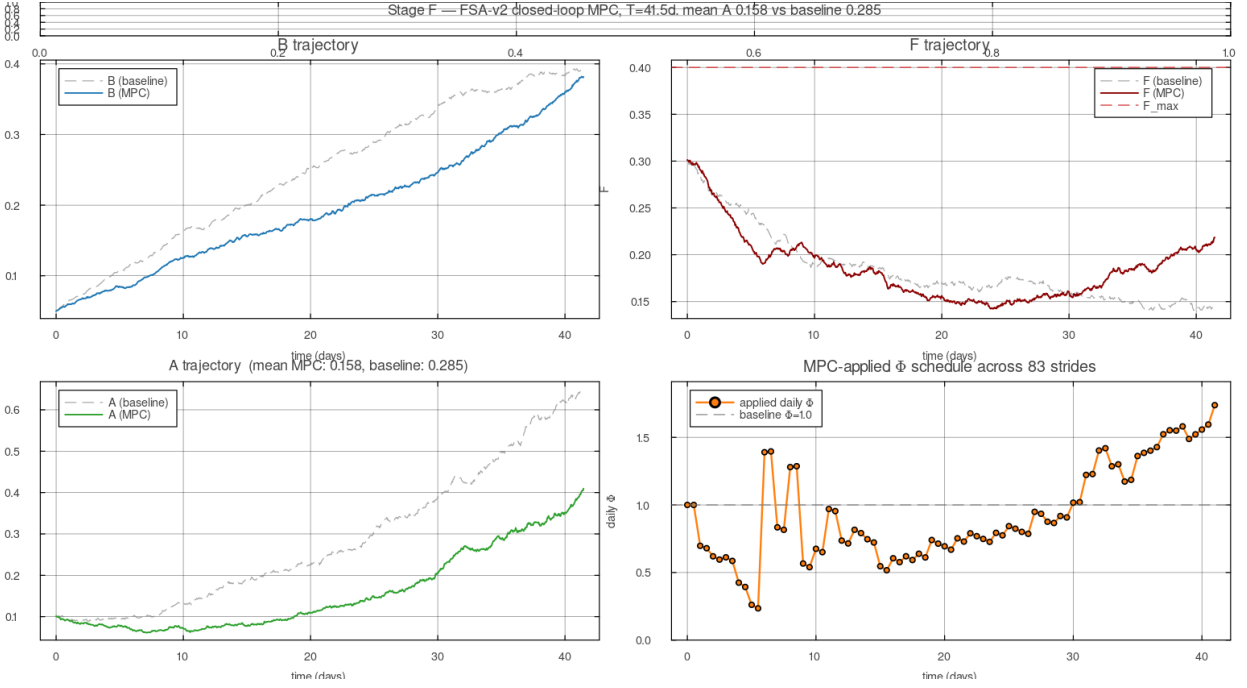


Figure 3: $T = 42$ days. State trajectories.

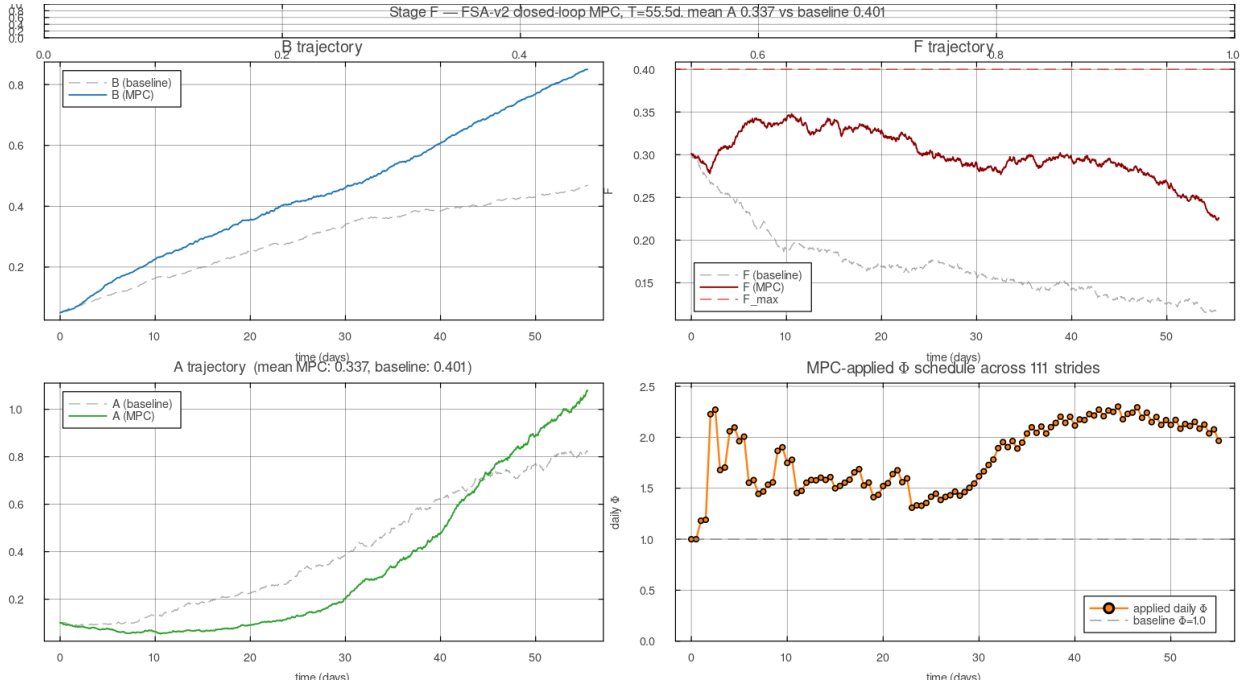


Figure 4: $T = 56$ days. State trajectories.

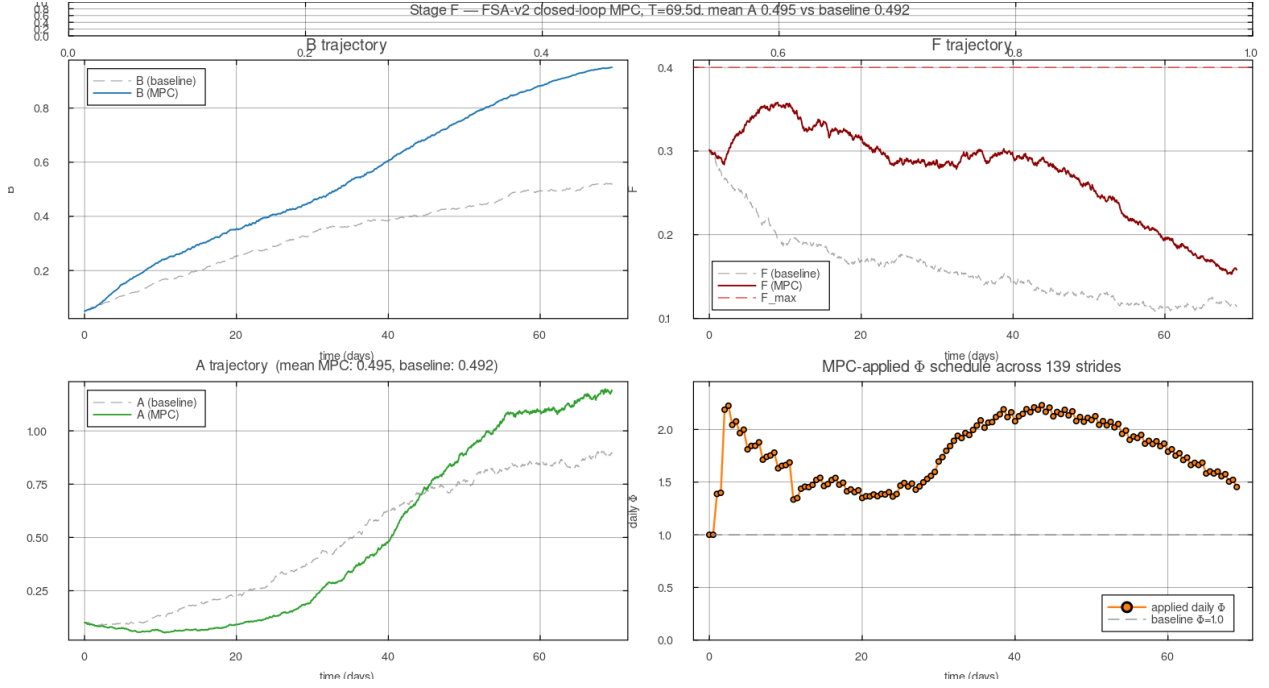


Figure 5: $T = 70$ days. State trajectories.

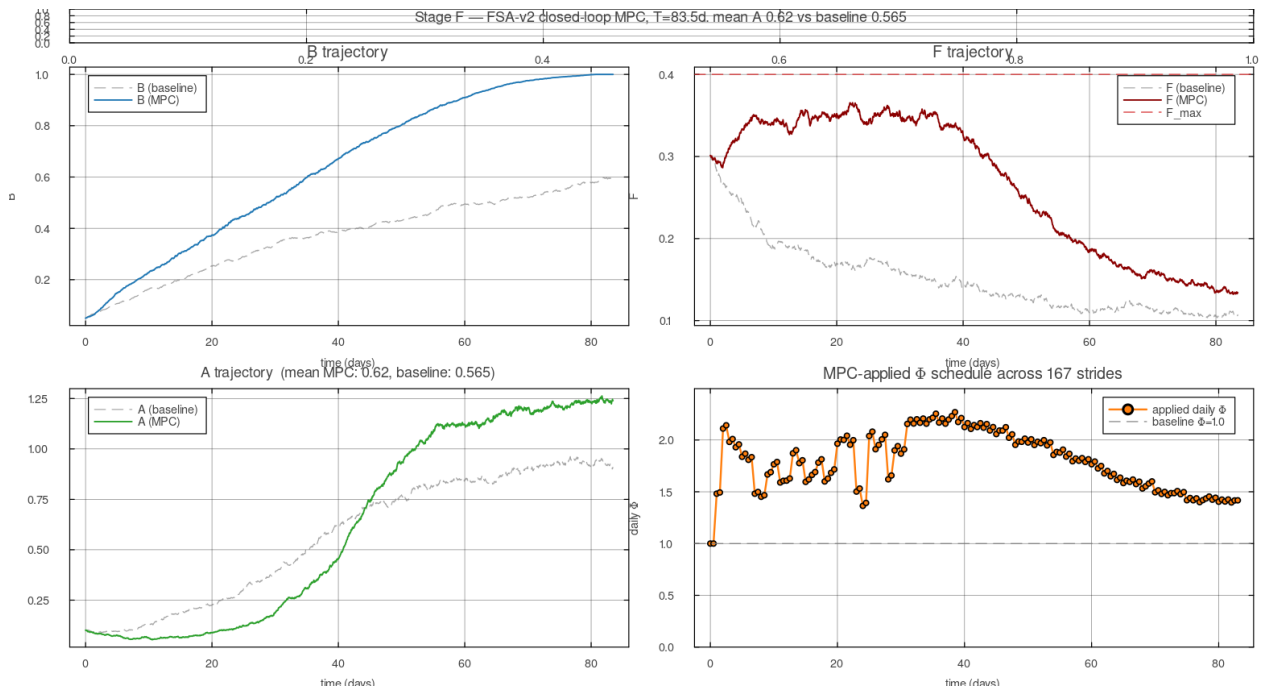


Figure 6: $T = 84$ days. State trajectories.

7 Posterior parameter traces

10-panel parameter-traces plot per horizon: per-parameter posterior median (blue line) plus 5–95% quantile band (light-blue fill) plus truth value (red dashed line), x -axis in days. Source: each horizon's `v15_T<N>d_param_traces.png`.

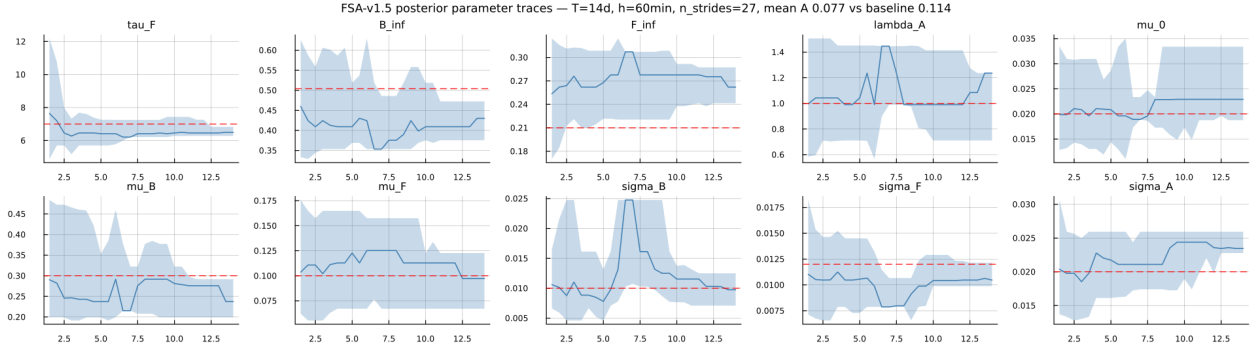


Figure 7: $T = 14$ days. Posterior parameter traces.

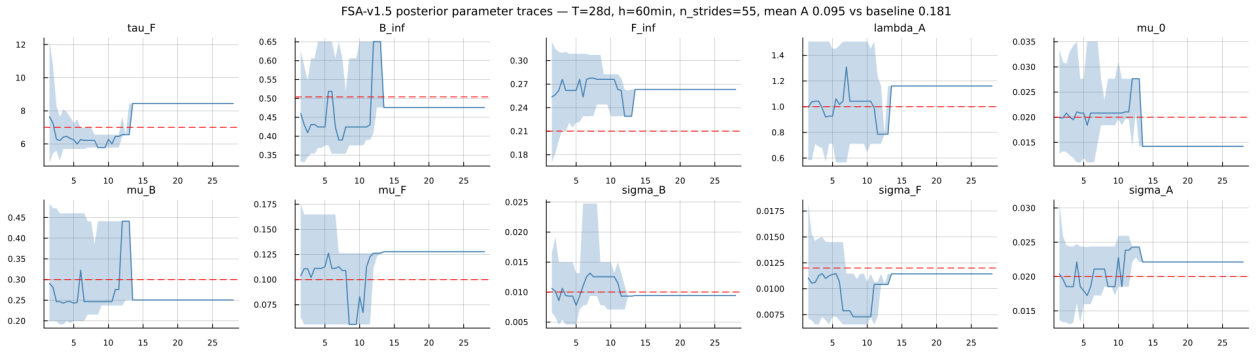


Figure 8: $T = 28$ days. Posterior parameter traces.

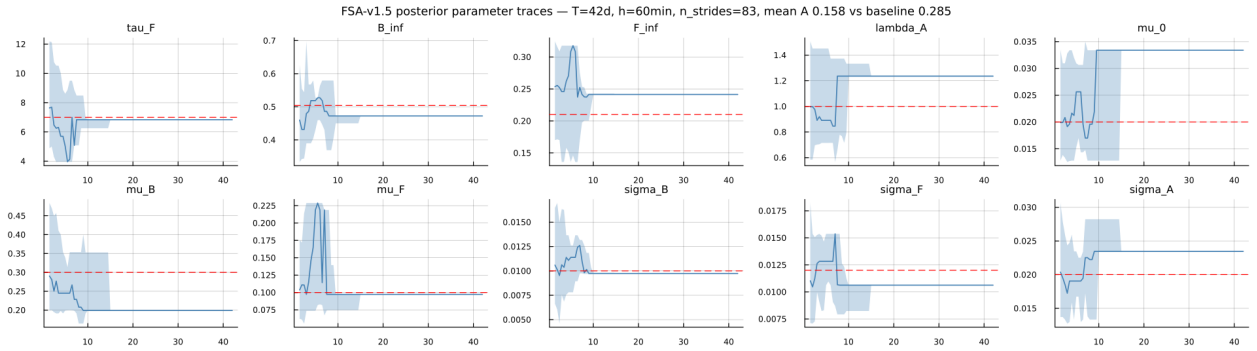


Figure 9: $T = 42$ days. Posterior parameter traces.

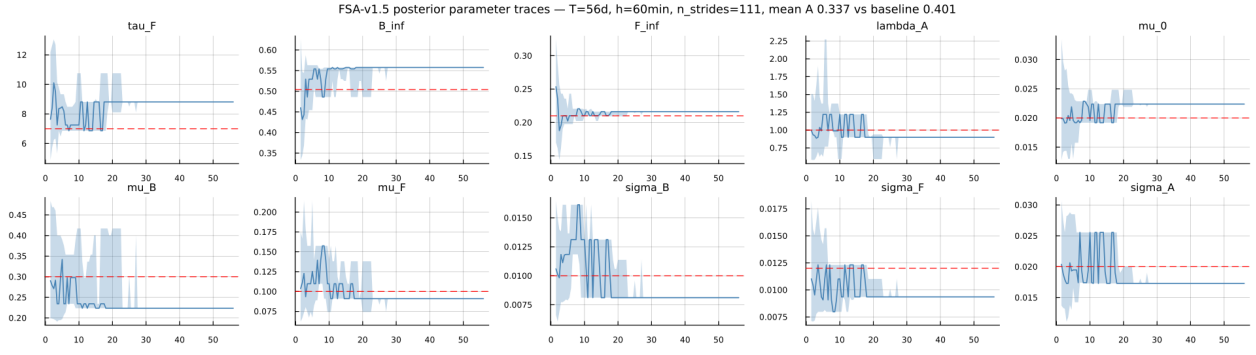


Figure 10: $T = 56$ days. Posterior parameter traces.

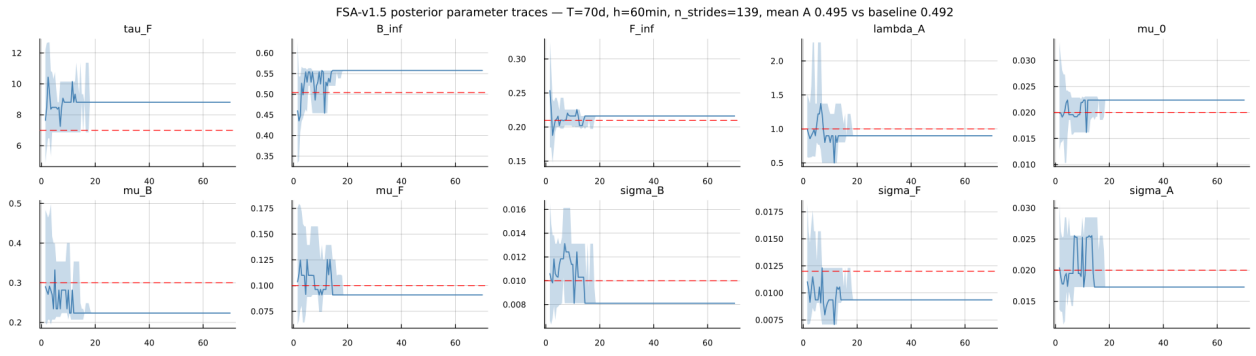


Figure 11: $T = 70$ days. Posterior parameter traces.

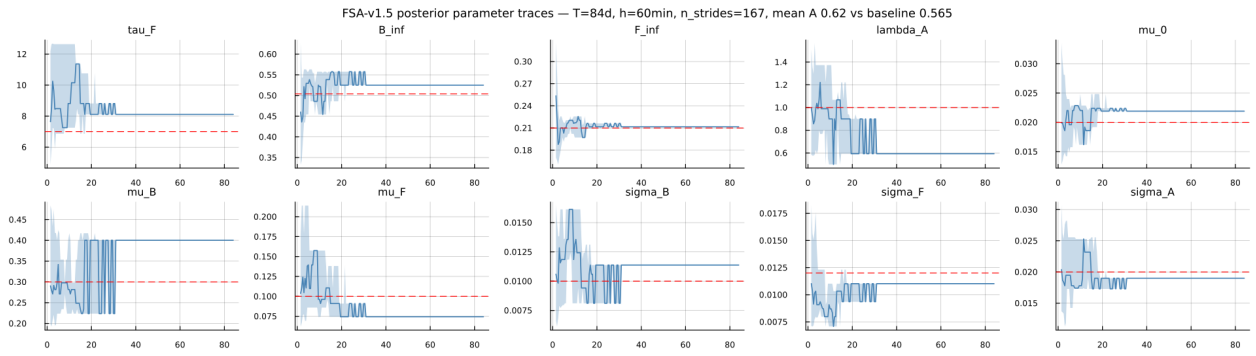


Figure 12: $T = 84$ days. Posterior parameter traces.

A Agent plan (verbatim)

The plan that drove this sweep, archived at `claude_plans/Julia_v1.5_multi_horizon_sweep_T_14_to_84_d.2026-`

```
# Julia v1.5 multi-horizon sweep (T = 14 84 d)

> Archived from plan mode: 2026-05-09 00:33.

## Context

We need a clean sweep of the Julia v1.5 closed-loop SMC-MPC bench at
six planning horizons T = 14, 28, 42, 56, 70, 84 days to study
how the controller's behaviour scales with horizon length on the
Julia side (the side we trust). All other config is the saturated
tier from
'compare_v15_julia_vs_python/docs/technical_guide_to_current_best_Julia_SMC2FC_config.pdf',
which is now the bench's *default* so the only flag that has to
change between runs is '--T-days'.

The work is:

1. **Quick T=14 run** (foreground, ~10 min) to confirm the launcher
  script is correct and the two PNGs (state traces + parameter
  traces) save into the expected place.
2. **Overnight chain T=28 T=84** (sequential, single-GPU, ~3.5 h
  total estimated from the 19.5-min T=28 wall in the tech guide).
3. Per-horizon artefacts land in a single sweep root with a
  well-named subdirectory per horizon.

## Tools to use

The user has pointed to the canonical plotting + profiling tools at
'compare_v15_julia_vs_python/src/':

* 'plot_state_traces.jl' 4-panel state-trajectory plot (B / F / A
  / applied) consumed via 'plot_state_traces(data; out_path=)'.
* 'plot_param_traces.jl' multi-panel posterior parameter traces
  (595 % envelope + median + red-dashed truth line) consumed via
  'plot_param_traces(data; out_path=, T_total_days=, step_minutes=,
  stride_bins=)'.
* 'profile_gpu_julia_v15.jl' micro-profiler that times the filter
  + controller kernels in isolation, reports kernel wall, threads in
  flight, FLOPs/call, effective TFLOPS vs the RTX 5090 fp32 peak, GPU
  memory, and host-sync count. Modes: '--filter', '--controller',
  '--both'.

The bench's own auto-plot fallback ('bench_smc_full_mpc_fsa_gpu.jl'
lines 840-854) tries to load 'version_2_Julia/tools/plot_state_traces.jl'
which is in the deprecated tree; it @warn's and continues on failure.
The launcher will call the canonical plotters at
'compare_v15_julia_vs_python/src/plot_*.jl' explicitly after each
horizon's bench finishes, so the sweep never depends on the bench's
internal auto-plot path.

The profiler is run ONCE at the start of the sweep kernel walls
do not depend on T-days (same chains inner-loop dimensions), only
n_steps and n_replans change, which the macro bench already records.

## What the Julia bench already gives us

Confirmed from 'version_1_5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl':

* All saturated-tier flags are now defaults (verified at lines 39-155
  of the bench): 'N-smc=512', 'K-per-chain=1000', 'num-mcmc=3',
  'max-temp-levels=30', 'ctrl-n-smc=2048', 'ctrl-num-mcmc=16',
  'ctrl-chees-max=512', 'ctrl-n-anchors=12', 'ctrl-max-levels=40',
  'replan-K=2', 'seed=42', 'step-minutes=60'. Default 'T-days=14'.
  So the only flag we vary is '--T-days'; we'll set '--seed 42'
  explicitly for reproducibility and '--output-dir' per horizon.

* Per run, the bench writes (lines 731-875):
  * 'data.jld2' (state + obs + posterior cloud)
  * 'manifest.json' (run config + replan history + total wall)
  * 'experiment_run.md'
  * 'per_stride.csv' (per-stride telemetry)
```

```

* 'v15_T<N>d_traces.png' (4-panel state + applied )
* 'v15_T<N>d_param_traces.png' (10-panel parameter posteriors)

We additionally tee stdout to a 'bench.log' per horizon.

* No existing Julia horizon-sweep launcher
('version_1_5_Julia/tools/launchers/' does not exist). The pattern
to mirror is 'version_2_Python_JAX/tools/launchers/run_horizon.sh'
(a single-T launcher), which we generalise to a multi-T loop.

## Estimated per-horizon walls (from tech guide T=28 19.5 min)

Linear scaling in 'n_replans T_days / 2' with the replan-K=2
default taken as a guess and to be replaced by the measured wall
in the final SWEEP_MANIFEST. Numbers below are the budget I expect:

| T_days | wall (guess) |
|---|---|
| 14 | ~10 min |
| 28 | ~20 min (anchor measured by Julia tech guide at 19.5 min) |
| 42 | ~30 min |
| 56 | ~40 min |
| 70 | ~50 min |
| 84 | ~60 min |
| **total** | **~3.5 h** |

These are guesses; the SWEEP_MANIFEST.md will record measured walls.

## Folder convention

'''
compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09/
SWEEP_MANIFEST.md          # rendered Markdown summary of horizons_results.csv
horizons_results.csv        # one row per horizon: full headline metrics table
microbench_profile.txt      # one-shot profile_gpu_julia_v15.jl --both output
T14d_seed42/
  data.jld2
  manifest.json
  experiment_run.md
  per_stride.csv            # per-stride wall, n_temp_filter/ctrl, daily_phi, A/B/F_end
  nvidia_smi.csv            # 1 Hz GPU util + memory samples for the run
  results.json              # extracted headline metrics for this horizon
  v15_T14d_traces.png       # canonical plot_state_traces.jl
  v15_T14d_param_traces.png # canonical plot_param_traces.jl
  bench.log                 # tee'd stdout
T28d_seed42/
  ...
T42d_seed42/
T56d_seed42/
T70d_seed42/
T84d_seed42/
'''

* Sweep root has a YYYY-MM-DD stamp so a future re-run does not
  collide.
* Per-horizon dir is 'T<N>d_seed<S>' readable by humans, sortable
  by horizon, and includes the seed so re-running at a different
  seed coexists.

## Headline metrics extracted per horizon

After each bench finishes, we run a small Python helper that reads
'data.jld2' (via 'h5py' Julia writes JLD2-on-HDF5), 'manifest.json',
'per_stride.csv', and 'nvidia_smi.csv', and writes:

* '<horizon>/results.json' all headline metrics for that horizon.
* one row appended to '<sweep_root>/horizons_results.csv'.

Metrics in the row (one per horizon):

| field | source |
|---|---|
| 'T_days' | sweep argument |
| 'n_strides' | 'manifest.json' |
| 'n_replans' | 'manifest.json' 'replan_history' length |

```

```
| 'total_wall_s' | 'manifest.json' 'total_elapsed_s' |
| 'mean_stride_wall_s' | 'per_stride.csv' mean of 't_wall_s' |
| 'mean_filter_n_temp' | 'per_stride.csv' mean of 'n_temp_filter' (non-zero rows) |
| 'mean_ctrl_n_temp' | 'per_stride.csv' mean of 'n_temp_ctrl' (non-zero rows) |
| 'mean_gpu_util_pct' | 'nvidia_smi.csv' mean of util column |
| 'peak_vram_mib' | 'nvidia_smi.csv' max of memory column |
| 'pct_time_gpu_above_90' | 'nvidia_smi.csv' fraction of samples 90 % |
| 'mean_A_mpc' | 'data.jld2' 'trajectory_mpc[:, 3]' mean |
| 'final_A_mpc' | 'data.jld2' 'trajectory_mpc[end, 3]' |
| 'mean_A_baseline' | 'data.jld2' 'trajectory_baseline[:, 3]' mean |
| 'final_A_baseline' | 'data.jld2' 'trajectory_baseline[end, 3]' |
| 'final_B_mpc' | 'data.jld2' 'trajectory_mpc[end, 1]' |
| 'final_F_mpc' | 'data.jld2' 'trajectory_mpc[end, 2]' |
| 'mean_phi_per_stride' | 'data.jld2' 'daily_phi_per_stride' mean |
| 'final_phi_per_stride' | 'data.jld2' 'daily_phi_per_stride[end]' |
| 'mean_phi_at_stride21' | the Julia tech guide's headline metric 'daily_phi_per_stride[21]' (skipped if '
    n_strides < 22') |
| 'bench_exit_code' | from the launcher loop |
```

Helper script: a small Python file at
 'compare_v15_julia_vs_python/src/extract_horizon_results.py' (new).
 Reads one horizon's directory, writes that horizon's 'results.json',
 and appends one row to the sweep-level 'horizons_results.csv'. We
 add 'from h5py import File' for the JLD2 reader (the existing repo
 uses 'h5py' for Julia Python crossover per the writeup).

After the sweep finishes, a final pass renders 'horizons_results.csv'
 into Markdown table form at the bottom of 'SWEEP_MANIFEST.md'.

Plan

Step 0 Write the launcher ('version_1_5_Julia/tools/launchers/run_julia_horizon_sweep.sh')

A single bash script that takes a list of T_days values and runs the
 bench sequentially. Mirrors the v2 Python 'run_horizon.sh' pattern
 but loops over multiple T's.

```
“‘bash
#!/usr/bin/env bash
# run_julia_horizon_sweep.sh [T1 T2 ]
# Default: 14 28 42 56 70 84
set -u
T_LIST=("${@:-14 28 42 56 70 84}")

REPO="$HOME/Repos/python-smc2-filtering-control"
SWEEP_DATE=$(date +%Y-%m-%d)
SWEEP_ROOT="$REPO/compare_v15_julia_vs_python/example_run/julia_horizon_sweep_${SWEEP_DATE}"
PLOT_DIR="$REPO/compare_v15_julia_vs_python/src"
mkdir -p "$SWEEP_ROOT"

cd "$REPO/version_1_5_Julia"

# Step 0: one-shot microbench profile (only if it doesn't already
# exist for this sweep date kernel walls don't depend on T-days).
PROFILE_OUT="$SWEEP_ROOT/microbench_profile.txt"
if [ ! -f "$PROFILE_OUT" ]; then
    echo "[$(date +%H:%M:%S)] one-shot microbench profile $PROFILE_OUT"
    julia --project=. \
        "$PLOT_DIR/profile_gpu_julia_v15.jl" --both 2>&1 | tee "$PROFILE_OUT"
fi

MANIFEST="$SWEEP_ROOT/SWEEP_MANIFEST.md"
{
    echo "# Julia v1.5 horizon sweep $(date)"
    echo
    echo "| T_days | start | wall_s | exit_code | dir |"
    echo "|---|---|---|---|---|"
} > "$MANIFEST"

for T in ${T_LIST[@]}; do
    DIR="$SWEEP_ROOT/T${T}d_seed42"
    mkdir -p "$DIR"
    LOG="$DIR/bench.log"
    SMI_CSV="$DIR/nvidia_smi.csv"
    START_TS=$(date +%Y-%m-%d %H:%M:%S)
```



```

T0=$(date +%s)
echo "[${START_TS}] T=${T}d $DIR"

# Start a 1 Hz nvidia-smi sampler in the background, kill it
# at end-of-bench. Captures GPU util %, used VRAM (MiB), and
# power draw per second so we can quote per-horizon mean util,
# peak VRAM, and time-spent-saturated.
{
    echo "timestamp,gpu_util_percent,memory_used_mib,power_w"
    while true; do
        nvidia-smi --query-gpu=timestamp,utilization.gpu,memory.used,power.draw \
            --format=csv,noheader,nounits \
            | awk -F', ' '{print $1,"$2","$3","$4}'
        sleep 1
    done
} > "$SMI_CSV" &
SMI_PID=$!

# Run the bench (writes data.jld2 + manifest.json + per_stride.csv)
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
    --T-days "$T" --seed 42 --output-dir "$DIR" \
    2>&1 | tee "$LOG"
rc=${PIPESTATUS[0]}

# Stop the GPU sampler.
kill "$SMI_PID" 2>/dev/null
wait "$SMI_PID" 2>/dev/null

# Force-call the canonical plotters on data.jld2 (the bench's
# internal auto-plot points at the deprecated v2 tree and may
# silently @warn).
if [ "$rc" -eq 0 ] && [ -f "$DIR/data.jld2" ]; then
    STATE_PNG="$DIR/v15_T${T}d_traces.png"
    PARAM_PNG="$DIR/v15_T${T}d_param_traces.png"
    julia --project=. -e "
        include(\"$PLOT_DIR/plot_state_traces.jl\")
        d = load_run_data(\"$DIR/data.jld2\")
        plot_state_traces(d; out_path=\"$STATE_PNG\")
        include(\"$PLOT_DIR/plot_param_traces.jl\")
        plot_param_traces(d; out_path=\"$PARAM_PNG\",
            T_total_days=$T, step_minutes=60, stride_bins=12)
    " 2>&1 | tee -a "$LOG"

    # Extract headline metrics results.json + horizons_results.csv
    python "$PLOT_DIR/extract_horizon_results.py" \
        --horizon-dir "$DIR" --T-days "$T" \
        --sweep-csv "$SWEEP_ROOT/horizons_results.csv" \
        --bench-exit-code "$rc" 2>&1 | tee -a "$LOG"
fi

T1=$(date +%s)
WALL=$((T1 - T0))
echo "| $T | $START_TS | $WALL | $rc | T${T}d_seed42 |" >> "$MANIFEST"
echo "[$(date '+%Y-%m-%d %H:%M:%S')] T=${T}d done in ${WALL}s exit=$rc"
done

# Render horizons_results.csv as a Markdown table at the bottom
# of SWEEP_MANIFEST.md so the sweep root is self-documenting.
if [ -f "$SWEEP_ROOT/horizons_results.csv" ]; then
{
    echo
    echo "## Per-horizon results"
    echo
    python -c "
import csv
with open('$SWEEP_ROOT/horizons_results.csv') as f:
    rows = list(csv.reader(f))
hdr, *body = rows
print('| ' + ' | '.join(hdr) + ' |')
print('| ' + ' | '.join(['---'] * len(hdr)) + ' |')
for r in body:
    print('| ' + ' | '.join(r) + ' |')
"
} >> "$MANIFEST"
fi

```

```

'''
Single small file, no dependencies beyond the bench + the canonical
plotters at 'compare_v15_julia_vs_python/src/'. Works for any subset
of horizons; default behaviour = full sweep. The microbench profile
is one-shot (skipped on rerun if 'microbench_profile.txt' already
exists in the sweep dir).

### Step 1 Quick verification at T=14 (~10 min, foreground)

'''bash
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
bash tools/launchers/run_julia_horizon_sweep.sh 14
'''

After it completes, verify by reading the directory listing:

'''bash
ls compare_v15_julia_vs_python/example_run/julia_horizon_sweep_<date>/T14d_seed42/
'''

Expect 6 files: 'data.jld2', 'manifest.json', 'experiment_run.md',
'per_stride.csv', 'v15_T14d_traces.png', 'v15_T14d_param_traces.png',
plus 'bench.log'.

Read both PNGs back and check:
* state traces show 28 strides (T=14 2 strides/day);
* param traces have 10 panels with truth lines.

If anything's off, fix the launcher before launching the long run.

### Step 2 Overnight chain T=28 T=84

'''bash
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
nohup bash tools/launchers/run_julia_horizon_sweep.sh 28 42 56 70 84 \
> /tmp/julia_horizon_sweep_$(date +%Y%m%d_%H%M).log 2>&1 &
'''

Detached via 'nohup', sequential (single GPU). Estimated ~3.5 h
total. The script writes per-T 'bench.log' files under each
horizon's directory; the master 'nohup' log captures the
launcher-level lines (start/done timestamps + per-T exit codes)
plus tee'd Julia stdout.

### Step 3 Post-sweep summary

Once Step 2 finishes:

* The launcher has already written 'SWEEP_MANIFEST.md' with the
  measured wall + exit code per horizon.
* For each horizon, read 'manifest.json' to extract:
  total_wall_s, n_strides, n_replans, last-stride applied, mean A
  vs baseline.
* Append a "Per-horizon results" table to 'SWEEP_MANIFEST.md' so
  the sweep root is self-documenting.

## Critical files

To create:
- 'version_1_5_Julia/tools/launchers/run_julia_horizon_sweep.sh'
  the sweep launcher with embedded GPU sampling + plot + extract
  steps.
- 'compare_v15_julia_vs_python/src/extract_horizon_results.py'
  small Python helper that reads a horizon's 'data.jld2' (via
  'h5py'), 'manifest.json', 'per_stride.csv', and 'nvidia_smi.csv'
  and writes both per-horizon 'results.json' and a sweep-level
  'horizons_results.csv' (creates with header on first call,
  appends row on subsequent calls).

To create at run time (the bench writes them):
- 'compare_v15_julia_vs_python/example_run/julia_horizon_sweep_<date>/SWEEP_MANIFEST.md'
- 'compare_v15_julia_vs_python/example_run/julia_horizon_sweep_<date>/T<N>d_seed42/...' 6

Read-only references:

```

```

- 'version_1_5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl' the
  bench (defaults already saturated; we only set '--T-days',
  '--seed', '--output-dir').
- 'compare_v15_julia_vs_python/docs/technical_guide_to_current_best_Julia_SMC2FC_config.pdf'
  config provenance.
- 'compare_v15_julia_vs_python/src/plot_state_traces.jl',
  'plot_param_traces.jl', 'profile_gpu_julia_v15.jl' canonical
  tools called explicitly by the launcher.
- 'version_2_Python_JAX/tools/launchers/run_horizon.sh' pattern
  to mirror for the bash structure.

## Verification

* **Step 1 gate (T=14 smoke):**
  * 'T14d_seed42/' contains 'data.jld2', 'manifest.json',
    'experiment_run.md', 'per_stride.csv', 'nvidia_smi.csv',
    'results.json', 'bench.log', 'v15_T14d_traces.png',
    'v15_T14d_param_traces.png'.
  * 'SWEEP_MANIFEST.md' has one row showing exit_code = 0.
  * 'microbench_profile.txt' exists in the sweep root.
  * 'horizons_results.csv' exists with one row of measured headline
    metrics (mean A_MPC, peak VRAM, mean GPU util, etc.).
  * Both PNGs render correctly when opened.
  * If any of the above is missing fix the launcher / extractor
    script before Step 2.

* **Step 2 gate (overnight chain):** every horizon directory has
  the same 6 artefacts; every row of 'SWEEP_MANIFEST.md' shows
  'exit_code = 0'. If any horizon failed, its 'bench.log' survives
  for postmortem (the loop continues to the next horizon on
  failure rather than aborting 'set -u' only catches unset vars,
  not non-zero exits).

* **Sanity:** 'T<N>d_traces.png' filenames match the horizon
  (e.g. 'T84d_seed42/v15_T84d_traces.png', not 'v15_T28d_traces.png').
  The plotter's output name is built from the '--T-days' value via
  the launcher's 'T${T}d' interpolation, so this is automatic, but
  worth eyeballing on the T=14 verification.

## Open questions

None at this point. The sweep set (14, 28, 42, 56, 70, 84) and seed
(42) match the user's note; everything else is the bench's
saturated-tier default per the tech guide.

```

References

- [1] *FSA v1.5 closed-loop SMC²-MPC: a side-by-side Python+JAX vs Julia comparison study*. 2026-05-08. [compare_v15_julia_vs_python/docs/julia_vs_python_v15_writeup.pdf](#).
- [2] *Current best Julia v1.5 SMC²-FC config (RTX 5090, T=28d)*. 2026-05-08. [compare_v15_julia_vs_python/docs/technical_guide_to_current_best_Julia_SMC2FC_config.pdf](#).